

Batch Size = 150

Epoch = 300

Time of training = about 8 hours

Training from Scratch (Without using pretrain weights)

Number of Test data = 10000

Number of Training data= 50000

مقدار رم GPU 4060 درگیر در حالت SNN حدودا 16 گیگ بوده ولی با همین تنظیمات در حالت ANN حجم اشغالی حدودا 23 گیگ بود.

Test: [0/45] eta: 0:09:37 loss: 0.2137 (0.2137) acc1: 95.5556 (95.5556) acc5: 99.5556 (99.5556) time: 12.8435 data: 11.7839
max mem: 3920

Test: [10/45] eta: 0:00:48 loss: 0.2137 (0.2350) acc1: 95.5556 (95.2323) acc5: 99.5556 (99.6768) time: 1.3763 data: 1.0714
max mem: 3920

Test: [20/45] eta: 0:00:20 loss: 0.2401 (0.2415) acc1: 94.6667 (94.9630) acc5: 99.5556 (99.7249) time: 0.2197 data: 0.0001
max mem: 3920

Test: [30/45] eta: 0:00:09 loss: 0.2405 (0.2407) acc1: 95.1111 (95.2545) acc5: 99.5556 (99.6703) time: 0.2074 data: 0.0001
max mem: 3920

Test: [40/45] eta: 0:00:02 loss: 0.2400 (0.2406) acc1: 95.5556 (95.3171) acc5: 99.5556 (99.6748) time: 0.1954 data: 0.0001
max mem: 3920

Test: [44/45] eta: 0:00:00 loss: 0.2400 (0.2414) acc1: 95.5556 (95.2900) acc5: 99.5556 (99.6800) time: 0.1899 data: 0.0001
max mem: 3920

Test: Total time: 0:00:22 (0.4941 s / it)

* Acc@1 95.290 Acc@5 99.680 loss 0.241

Accuracy of the network on 10000 test images: 95.29000%

نتایج بررسی Sparsity :

Batch [1/67] Accuracy: 96.00% | Avg Layer Sparsity: 33.20%
Batch [10/67] Accuracy: 95.00% | Avg Layer Sparsity: 33.22%
Batch [20/67] Accuracy: 95.03% | Avg Layer Sparsity: 33.33%
Batch [30/67] Accuracy: 94.96% | Avg Layer Sparsity: 32.98%
Batch [40/67] Accuracy: 95.18% | Avg Layer Sparsity: 33.33%
Batch [50/67] Accuracy: 95.29% | Avg Layer Sparsity: 33.18%
Batch [60/67] Accuracy: 95.32% | Avg Layer Sparsity: 33.31%

=====

FINAL EVALUATION RESULTS For SPARSITY

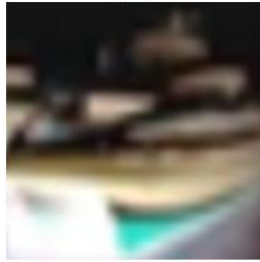
=====

Overall Accuracy: 95.29%
Overall Average Sparsity: 0.33%
Total samples evaluated: 7500
Per-Layer Average Sparsities:
 downsample_layers: 0.00%
 head: 99.64%
 stages: 0.00%

True: cat (#3)
Pred: cat (#3) 0.92



True: ship (#8)
Pred: ship (#8) 0.93



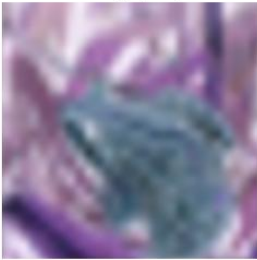
True: ship (#8)
Pred: ship (#8) 0.52



True: airplane (#0)
Pred: airplane (#0) 0.92



True: frog (#6)
Pred: frog (#6) 0.93



True: frog (#6)
Pred: frog (#6) 0.92



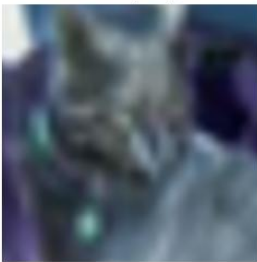
True: automobile (#1)
Pred: automobile (#1) 0.92



True: frog (#6)
Pred: frog (#6) 0.87



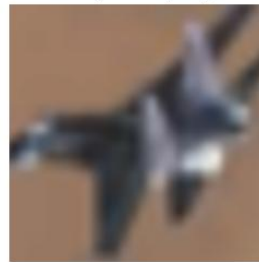
True: cat (#3)
Pred: cat (#3) 0.92



True: automobile (#1)
Pred: automobile (#1) 0.88



True: airplane (#0)
Pred: airplane (#0) 0.83



True: truck (#9)
Pred: truck (#9) 0.92



True: dog (#5)
Pred: dog (#5) 0.84



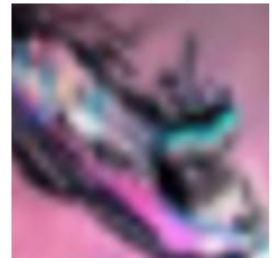
True: horse (#7)
Pred: horse (#7) 0.95



True: truck (#9)
Pred: truck (#9) 0.92



True: ship (#8)
Pred: ship (#8) 0.90



Layer (type:depth-idx)	Output Shape	Param #
ConvNeXtSpiking	[1, 10]	--
└ModuleList: 1-7	--	(recursive)
└Sequential: 2-1	[1, 96, 8, 8]	--
└Conv2d: 3-1	[1, 96, 8, 8]	4,704
└ModuleList: 1-8	--	(recursive)
└Sequential: 2-2	[1, 96, 8, 8]	--
└SpikingBlock: 3-2	[1, 96, 8, 8]	79,104
└SpikingBlock: 3-3	[1, 96, 8, 8]	79,104
└SpikingBlock: 3-4	[1, 96, 8, 8]	79,104
└ModuleList: 1-7	--	(recursive)
└Sequential: 2-3	[1, 192, 4, 4]	--
└Conv2d: 3-5	[1, 192, 4, 4]	73,920
└ModuleList: 1-8	--	(recursive)
└Sequential: 2-4	[1, 192, 4, 4]	--
└SpikingBlock: 3-6	[1, 192, 4, 4]	305,664
└SpikingBlock: 3-7	[1, 192, 4, 4]	305,664
└SpikingBlock: 3-8	[1, 192, 4, 4]	305,664
└ModuleList: 1-7	--	(recursive)
└Sequential: 2-5	[1, 384, 2, 2]	--
└Conv2d: 3-9	[1, 384, 2, 2]	295,296
└ModuleList: 1-8	--	(recursive)
└Sequential: 2-6	[1, 384, 2, 2]	--
└SpikingBlock: 3-10	[1, 384, 2, 2]	1,201,152
└SpikingBlock: 3-11	[1, 384, 2, 2]	1,201,152
└SpikingBlock: 3-12	[1, 384, 2, 2]	1,201,152
└SpikingBlock: 3-13	[1, 384, 2, 2]	1,201,152
└SpikingBlock: 3-14	[1, 384, 2, 2]	1,201,152
└SpikingBlock: 3-15	[1, 384, 2, 2]	1,201,152
└SpikingBlock: 3-16	[1, 384, 2, 2]	1,201,152
└SpikingBlock: 3-17	[1, 384, 2, 2]	1,201,152
└SpikingBlock: 3-18	[1, 384, 2, 2]	1,201,152
└ModuleList: 1-7	--	(recursive)
└Sequential: 2-7	[1, 768, 1, 1]	--
└Conv2d: 3-19	[1, 768, 1, 1]	1,180,416
└ModuleList: 1-8	--	(recursive)
└Sequential: 2-8	[1, 768, 1, 1]	--
└SpikingBlock: 3-20	[1, 768, 1, 1]	4,761,600
└SpikingBlock: 3-21	[1, 768, 1, 1]	4,761,600
└SpikingBlock: 3-22	[1, 768, 1, 1]	4,761,600
└Linear: 1-9	[1, 10]	7,690
Trainable params: 27,811,498		
Non-trainable params: 0		
Total mult-adds (M): 6.04		
Input size (MB): 0.01		
Forward/backward pass size (MB): 0.44		
Params size (MB): 7.57		
Estimated Total Size (MB): 8.03		

```

Model = ConvNeXtSpiking(
  (downsample_layers): ModuleList(
    (0): Sequential(
      (0): Conv2d(3, 96, kernel_size=(4, 4), stride=(4, 4))
    )
    (1): Sequential(
      (0): Conv2d(96, 192, kernel_size=(2, 2), stride=(2, 2))
    )
    (2): Sequential(
      (0): Conv2d(192, 384, kernel_size=(2, 2), stride=(2, 2))
    )
    (3): Sequential(
      (0): Conv2d(384, 768, kernel_size=(2, 2), stride=(2, 2))
    )
  )
  (stages): ModuleList(
    (0): Sequential(
      (0): SpikingBlock(
        (dwconv): Conv2d(96, 96, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=96)
        (pw1): Linear(in_features=96, out_features=384, bias=True)
        (pw2): Linear(in_features=384, out_features=96, bias=True)
        (drop_path): Identity()
      )
      (1): SpikingBlock(
        (dwconv): Conv2d(96, 96, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=96)
        (pw1): Linear(in_features=96, out_features=384, bias=True)
        (pw2): Linear(in_features=384, out_features=96, bias=True)
        (drop_path): Identity()
      )
      (2): SpikingBlock(
        (dwconv): Conv2d(96, 96, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=96)

```

```

    (pw1): Linear(in_features=96, out_features=384, bias=True)
    (pw2): Linear(in_features=384, out_features=96, bias=True)
    (drop_path): Identity()
)
)
(1): Sequential(
  (0): SpikingBlock(
    (dwconv): Conv2d(192, 192, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=192)
    (pw1): Linear(in_features=192, out_features=768, bias=True)
    (pw2): Linear(in_features=768, out_features=192, bias=True)
    (drop_path): Identity()
  )
  (1): SpikingBlock(
    (dwconv): Conv2d(192, 192, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=192)
    (pw1): Linear(in_features=192, out_features=768, bias=True)
    (pw2): Linear(in_features=768, out_features=192, bias=True)
    (drop_path): Identity()
  )
  (2): SpikingBlock(
    (dwconv): Conv2d(192, 192, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=192)
    (pw1): Linear(in_features=192, out_features=768, bias=True)
    (pw2): Linear(in_features=768, out_features=192, bias=True)
    (drop_path): Identity()
  )
)
(2): Sequential(
  (0): SpikingBlock(
    (dwconv): Conv2d(384, 384, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=384)
    (pw1): Linear(in_features=384, out_features=1536, bias=True)
    (pw2): Linear(in_features=1536, out_features=384, bias=True)
    (drop_path): Identity()
  )

```

```

)

(1): SpikingBlock(
  (dwconv): Conv2d(384, 384, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=384)
  (pw1): Linear(in_features=384, out_features=1536, bias=True)
  (pw2): Linear(in_features=1536, out_features=384, bias=True)
  (drop_path): Identity()
)

(2): SpikingBlock(
  (dwconv): Conv2d(384, 384, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=384)
  (pw1): Linear(in_features=384, out_features=1536, bias=True)
  (pw2): Linear(in_features=1536, out_features=384, bias=True)
  (drop_path): Identity()
)

(3): SpikingBlock(
  (dwconv): Conv2d(384, 384, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=384)
  (pw1): Linear(in_features=384, out_features=1536, bias=True)
  (pw2): Linear(in_features=1536, out_features=384, bias=True)
  (drop_path): Identity()
)

(4): SpikingBlock(
  (dwconv): Conv2d(384, 384, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=384)
  (pw1): Linear(in_features=384, out_features=1536, bias=True)
  (pw2): Linear(in_features=1536, out_features=384, bias=True)
  (drop_path): Identity()
)

(5): SpikingBlock(
  (dwconv): Conv2d(384, 384, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=384)
  (pw1): Linear(in_features=384, out_features=1536, bias=True)
  (pw2): Linear(in_features=1536, out_features=384, bias=True)
  (drop_path): Identity()
)

```

```
(6): SpikingBlock(  
  (dwconv): Conv2d(384, 384, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=384)  
  (pw1): Linear(in_features=384, out_features=1536, bias=True)  
  (pw2): Linear(in_features=1536, out_features=384, bias=True)  
  (drop_path): Identity()  
)
```

```
(7): SpikingBlock(  
  (dwconv): Conv2d(384, 384, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=384)  
  (pw1): Linear(in_features=384, out_features=1536, bias=True)  
  (pw2): Linear(in_features=1536, out_features=384, bias=True)  
  (drop_path): Identity()  
)
```

```
(8): SpikingBlock(  
  (dwconv): Conv2d(384, 384, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=384)  
  (pw1): Linear(in_features=384, out_features=1536, bias=True)  
  (pw2): Linear(in_features=1536, out_features=384, bias=True)  
  (drop_path): Identity()  
)  
)
```

```
(3): Sequential(  
  (0): SpikingBlock(  
    (dwconv): Conv2d(768, 768, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=768)  
    (pw1): Linear(in_features=768, out_features=3072, bias=True)  
    (pw2): Linear(in_features=3072, out_features=768, bias=True)  
    (drop_path): Identity()  
  )  
  (1): SpikingBlock(  
    (dwconv): Conv2d(768, 768, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=768)  
    (pw1): Linear(in_features=768, out_features=3072, bias=True)  
    (pw2): Linear(in_features=3072, out_features=768, bias=True)  
    (drop_path): Identity()  
  )  
)
```



```
)  
(2): SpikingBlock(  
  (dwconv): Conv2d(768, 768, kernel_size=(7, 7), stride=(1, 1), padding=(3, 3), groups=768)  
  (pw1): Linear(in_features=768, out_features=3072, bias=True)  
  (pw2): Linear(in_features=3072, out_features=768, bias=True)  
  (drop_path): Identity()  
)  
)  
)  
(head): Linear(in_features=768, out_features=10, bias=True)  
)
```

number of params: 27811498

LR = 0.00040000

Batch size = 150

Update frequent = 1

Number of training examples = 50000

Number of Epoch: 300

Number of training training per epoch (iteration) = 333